Estándares de Frontend — Vue 3 + Quasar

Fecha: 2026-04-11
Aplica a: Todo código en `ttpn-frontend/src/`

> El documento detallado con ejemplos completos de código está en `ttpn-frontend/documentacion/ESTANDAR_DESARROLLO_FRONTEND.md`.
> Este documento es la referencia rápida de reglas y decisiones.

---

1. Patrón Obligatorio de Página

Toda pantalla sigue esta estructura. No hay excepciones.

```
Page.vue (orquestador — máx 300 líneas)
├── usa composables para toda la lógica
├── pasa datos a componentes via props
└── components/
    ├── XxxTable.vue          ← tabla desktop con AppTable
    ├── XxxMobileList.vue     ← lista móvil (q-list)
    ├── XxxFilters.vue        ← campos del panel de filtros
    ├── XxxForm.vue           ← formulario en dialog
    └── XxxDetailsDialog.vue  ← vista de detalle (si aplica)
```

Si la Page tiene más de 300 líneas, algo de lógica debe moverse a un composable.

---

2. Composables — Responsabilidades

Cada composable tiene una sola responsabilidad. No mezclar.

Composable	Responsabilidad
`useXxxData.js`	Fetch, paginación, lista de registros
`useXxxCatalogs.js`	Dropdowns que necesita el módulo
`useXxxForm.js`	Estado del form, save, validaciones
`useFilters.js`	Estado de filtros, contador de activos, clear
`useCrud.js`	CRUD genérico reutilizable (para catálogos simples)
`usePrivileges.js`	Verificación de permisos por módulo
`useNotify.js`	Notificaciones toast
`useDateFormat.js`	Formateo de fechas

Nombrado: siempre `use` + PascalCase. El archivo: `useMiComposable.js`.

Cuándo usar `useCrud` vs composable propio

```
Módulo simple (catálogo: nombre + status) → useCrud directamente
Módulo con lógica extra (filtros, acciones especiales) → composable propio
```

---

3. Servicios API

Un archivo por dominio en `src/services/`. Exporta un objeto plano con métodos.

```
// src/services/vehicles.service.js
import { api } from 'boot/axios'

export const vehiclesService = {
  list:    (params) => api.get('/api/v1/vehicles', { params }),
  find:    (id)     => api.get(`/api/v1/vehicles/${id}`),
  create:  (data)   => api.post('/api/v1/vehicles', data),
  update:  (id, data) => api.put(`/api/v1/vehicles/${id}`, data),
  destroy: (id)     => api.delete(`/api/v1/vehicles/${id}`),
  // Acciones extra:
  activate:   (id) => api.patch(`/api/v1/vehicles/${id}/activate`),
  deactivate: (id) => api.patch(`/api/v1/vehicles/${id}/deactivate`),
}
```

Reglas:

- Nunca llamar a `api` directamente desde una Page o composable — siempre pasar por el service
- El wrapper del payload lo construye el composable: `{ vehicle: form.value }`
- Usar `api.put` para updates completos, `api.patch` para acciones parciales (activate, etc.)

---

4. Manejo de Errores

Una sola función para todos los errores de API:

```
import { useNotify } from 'src/composables/useNotify'
const { notifyOk, notifyError, notifyApiError } = useNotify()

// Error de API (4xx, 5xx):
try {
  await service.create(payload)
  notifyOk('Creado correctamente')
} catch (error) {
  notifyApiError(error, 'Error al crear')
  // notifyApiError extrae error.response.data.errors.join(', ')
  // o muestra el fallback si no hay errors array
}
```

Nunca:

```
// ❌ No hacer esto
catch (e) {
  console.error(e)  // sin notificación al usuario
}

// ❌ No hacer esto
catch (e) {
  alert(e.message)
}

// ❌ No hacer esto — mensaje hardcodeado que ignora el error del BE
catch (e) {
  notifyError('Ocurrió un error')
}
```

---

5. Control de Acceso por Privilegios

Nunca verificar permisos con `role_id` hardcodeado. Siempre usar `usePrivileges`.

```
// En el composable o en la Page:
const { canCreate, canEdit, canDelete, canExport } =
usePrivileges('vehicles')

// En el template:
// <q-btn v-if="canCreate()" @click="openDialog()" label="Nuevo" />
// <q-btn v-if="canDelete()" @click="confirmDelete(row)" label="Eliminar"
/>
```

El `moduleKey` es el campo `module_key` del modelo `Privilege` en el BE.

---

6. Paginación

```
// Estado estándar de paginación
const pagination = ref({
  page: 1,
  rowsPerPage: 20,
  rowsNumber: 0   // total del BE
})

// Al recibir respuesta del BE:
const { data } = await service.list({ page: pagination.value.page,
per_page: pagination.value.rowsPerPage })
items.value = data.data           // o data.versions, según el endpoint
```

```
pagination.value.rowsNumber = data.meta.total_count

// Al cambiar página (evento de AppTable o q-table):
async function onPageChange(newPagination) {
  pagination.value = newPagination
  await fetchData()
}
```

---

7. Estados de Carga

```
const loading = ref(false)
const saving  = ref(false)

// loading: mientras se carga la lista
// saving: mientras se guarda/elimina un registro

async function fetchData() {
  loading.value = true
  try { /* ... */ } finally { loading.value = false }
}

async function save() {
  saving.value = true
  try { /* ... */ } finally { saving.value = false }
}
```

En el template:

```
<AppTable :loading="loading" ... />
<q-btn :loading="saving" label="Guardar" type="submit" />
```

---

8. Formularios y Dialogs

```
// Estado del dialog
const showDialog = ref(false)
const editingItem = ref(null)
const form = ref({ ...formDefault })

function openCreate() {
  editingItem.value = null
  form.value = { ...formDefault }
  showDialog.value = true
}

function openEdit(item) {
```

```
  editingItem.value = item
  form.value = { ...item }        // shallow copy — no mutar el original
  showDialog.value = true
}
```

Reglas de formularios:

- El form siempre es una copia (`{ ...item }`), nunca referencia directa al objeto de la lista
- El payload al BE siempre con wrapper: `{ recurso: form.value }`
- Cerrar el dialog con `showDialog.value = false` SOLO si el save fue exitoso (en el `try`, antes del `finally`)
- Después de guardar, siempre hacer `fetchData()` para refrescar la lista

---

9. Filtros

```
import { useFilters } from 'src/composables/useFilters'

const { filters, showFilters, activeFiltersCount, clearFilters,
toggleFilters } = useFilters({
  fecha_inicio: null,
  fecha_fin:    null,
  status:       null,
  client_id:    null,
})

// Al aplicar filtros: watch + fetchData
watch(filters, () => { fetchData() }, { deep: true })

// Al limpiar:
function onClearFilters() {
  clearFilters()
  fetchData()
}
```

En el template, el botón de filtros siempre muestra el badge:

```
<q-btn @click="toggleFilters">
  Filtros
  <q-badge v-if="activeFiltersCount > 0" floating color="red">
    {{ activeFiltersCount }}
  </q-badge>
</q-btn>
```

---

10. Responsive

Todas las páginas soportan desktop y móvil. Patrón:

```
<!-- Desktop: tabla -->
<XxxTable v-if="$q.screen.gt.sm" :items="items" ... />

<!-- Móvil: lista de cards -->
<XxxMobileList v-else :items="items" ... />
```

En los dialogs usar `$q.screen.lt.md` para ajustar el ancho:

```
<q-dialog v-model="showDialog">
  <q-card :style="$q.screen.lt.md ? 'width:100%' : 'min-width:600px'">
```

---

11. Confirmaciones de Eliminación

Siempre usar `$q.dialog` para confirmar antes de eliminar:

```
function confirmDelete(item) {
  $q.dialog({
    title: 'Confirmar eliminación',
    message: `¿Eliminar "${item.nombre}"? Esta acción no se puede
deshacer.`,
    cancel: true,
    persistent: true
  }).onOk(async () => {
    try {
      await service.destroy(item.id)
      notifyOk('Eliminado correctamente')
      await fetchData()
    } catch (e) {
      notifyApiError(e, 'Error al eliminar')
    }
  })
}
```

---

12. Nomenclatura

Cosa	Convención	Ejemplo
Archivos de página	PascalCase + Page	`EmployeesPage.vue`
Archivos de componente	PascalCase	`EmployeeForm.vue`
Composables	camelCase + use	`useEmployeeData.js`

Cosa	Convención	Ejemplo
Servicios	camelCase + Service	`employees.service.js`
Variables reactivas	camelCase	`const items = ref([])`
Eventos emitidos	kebab-case	`emit('item-saved')`
Props	camelCase en JS, kebab en HTML	`:item-id="id"`

13. Lo que NO hacer

```
// ❌ role_id hardcodeado para control de acceso
if (user.role_id === 1) { ... }
// ✅ usar canCreate(), canEdit(), etc.

// ❌ llamar a la API directamente en el template o en una Page
const { data } = await api.get('/api/v1/employees')
// ✅ siempre a través de service y composable

// ❌ mutar props directamente
props.item.nombre = 'nuevo'
// ✅ emitir evento al padre o copiar con { ...props.item }

// ❌ lógica de negocio en el template (expresiones largas en v-if)
v-if="items.filter(i => i.status).length > 0 && user.role_id !== 2"
// ✅ computed en el composable

// ❌ console.log en código que llega a producción
// El linter de SonarCloud lo marca como issue
```